

API_GUIDELINES.md

Version: 1.0

Status: Mandatory

Classification: API Engineering Standard

Applies To

- Backend APIs
 - Frontend API Calls
 - External Integrations
 - Webhooks
 - Internal Services
-

1. Purpose

This document defines the API standards for CommercePilot.

Every API must follow these rules.

Consistency is mandatory.

2. API Philosophy

Every API should be:

- Predictable
 - Versioned
 - RESTful
 - Secure
 - Consistent
 - Documented
 - Testable
 - Backward Compatible
-

3. API Base URL

Development

/api/v1

Example

/api/v1/orders

/api/v1/customers

/api/v1/products

Never expose APIs without versioning.

4. Naming Convention

Use plural resource names.

Correct

/orders

/products

/customers

/messages

Incorrect

/getOrders

/createOrder

/productList

Use lowercase.

Use hyphens only when necessary.

5. HTTP Methods

GET

Retrieve resources

POST

Create resources

PUT

Replace resources

PATCH

Partial updates

DELETE

Soft delete only unless explicitly documented.

6. Standard Response Format

Success

```
{ "success": true, "message": "Order created successfully.", "data": {}, "meta": {} }
```

Failure

```
{ "success": false, "message": "Validation failed.", "errors": [] }
```

Never return inconsistent response structures.

7. HTTP Status Codes

200 OK

201 Created

204 No Content

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

422 Validation Error

429 Too Many Requests

500 Internal Server Error

Never return 200 for failures.

8. Pagination

Large collections must use pagination.

Request

GET /orders?page=1&limit=20

Response

```
{ "success": true, "data": [], "meta": { "page": 1, "limit": 20, "total": 250, "totalPages": 13 } }
```

Never return thousands of records in one request.

9. Filtering

Supported format

/orders?status=pending

/products?category=shirts

/customers?active=true

Multiple filters allowed.

10. Sorting

Example

/orders?sort=createdAt

Descending

/orders?sort=-createdAt

Multiple sorting

/orders?sort=status,-createdAt

11. Searching

Use

search=

Example

/products?search=school shirt

Customers

/customers?search=nimal

12. Field Selection

Allow

fields=

Example

/orders?fields=id,status,total

Improve performance by returning only requested fields.

13. Validation

Validate

UUID

Email

Phone

Quantity

Price

Dates

Enums

Never trust client input.

14. Error Response

Standard

```
{ "success": false, "message": "Validation failed.", "errors": [ { "field": "email", "message": "Invalid email address." } ] }
```

Never expose stack traces.

15. Authentication

JWT required.

Authorization Header

Bearer <token>

Never accept authentication in query parameters.

16. Authorization

Every endpoint must verify:

Authentication

Role

Tenant

Permissions

Frontend checks never replace backend authorization.

17. Multi-Tenant Rules

Every request belongs to exactly one tenant.

Every database query must filter by tenant_id.

No endpoint may return another tenant's data.

18. Idempotency

Required for:

Order creation

Payment callbacks

Webhook processing

Support

Idempotency-Key

header where appropriate.

Prevent duplicate processing.

19. Webhook Standards

Always verify signatures.

Log webhook events.

Return HTTP 200 only after successful validation.

Retry failed processing using queues.

Never process webhooks synchronously if long-running.

20. Rate Limiting

Apply limits to:

Authentication

Public APIs

AI endpoints

Webhook endpoints

Sensitive operations

Return

429

when exceeded.

21. API Versioning

Current

v1

Future

v2

Never introduce breaking changes without a new version.

22. Documentation

Every endpoint requires:

Summary

Description

Parameters

Authentication

Request Example

Response Example

Error Responses

Swagger/OpenAPI documentation is mandatory.

23. Logging

Log

Request ID

User ID

Tenant ID

Endpoint

Method

Duration

Status

Never log secrets or tokens.

24. Performance

Enable pagination.

Avoid N+1 queries.

Select only required fields.

Use indexes.

Queue expensive operations.

Cache read-heavy endpoints when appropriate.

25. External Integrations

Wrap all third-party APIs in adapter services.

Never expose external API responses directly.

Normalize responses before returning them.

Supported integrations include:

- WhatsApp Cloud API
 - WooCommerce REST API
 - Email provider
 - AI provider
-

26. API Design Rules

Controllers

Receive request

Validate input

Call service

Return response

Controllers must not contain business logic.

Services

Contain application logic.

Repositories

Contain database access only.

External APIs

Must be accessed through integration adapters.

27. Endpoint Naming Examples

GET

/orders

GET

/orders/{id}

POST

/orders

PATCH

/orders/{id}

DELETE

/orders/{id}

GET

/orders/{id}/history

GET

/customers/{id}/orders

Maintain nested resources only where relationships are clear.

28. OpenAPI Standards

Every endpoint must include:

- Summary
- Description
- Tags
- Parameters
- Request schema
- Response schema
- Example payloads
- Error responses
- Authentication requirements

Swagger documentation must remain synchronized with implementation.

29. Testing Requirements

Every endpoint requires:

- Unit tests
- Integration tests
- Authorization tests
- Validation tests
- Tenant isolation tests

Critical endpoints should also include idempotency and retry tests.

30. API Quality Checklist

Before an endpoint is complete:

- ✓ RESTful design
 - ✓ Versioned URL
 - ✓ Authentication enforced
 - ✓ Authorization enforced
 - ✓ Tenant isolation verified
 - ✓ Input validated
 - ✓ Standard response format
 - ✓ Proper status codes
 - ✓ Pagination implemented (where required)
 - ✓ Error handling implemented
 - ✓ Logging added
 - ✓ Swagger updated
 - ✓ Tests passing
-

Final Rule

Every API must be designed as if it will be consumed by third-party developers.

Consistency is more valuable than cleverness.

Breaking standards for convenience is prohibited.

Production quality is the minimum acceptable standard.